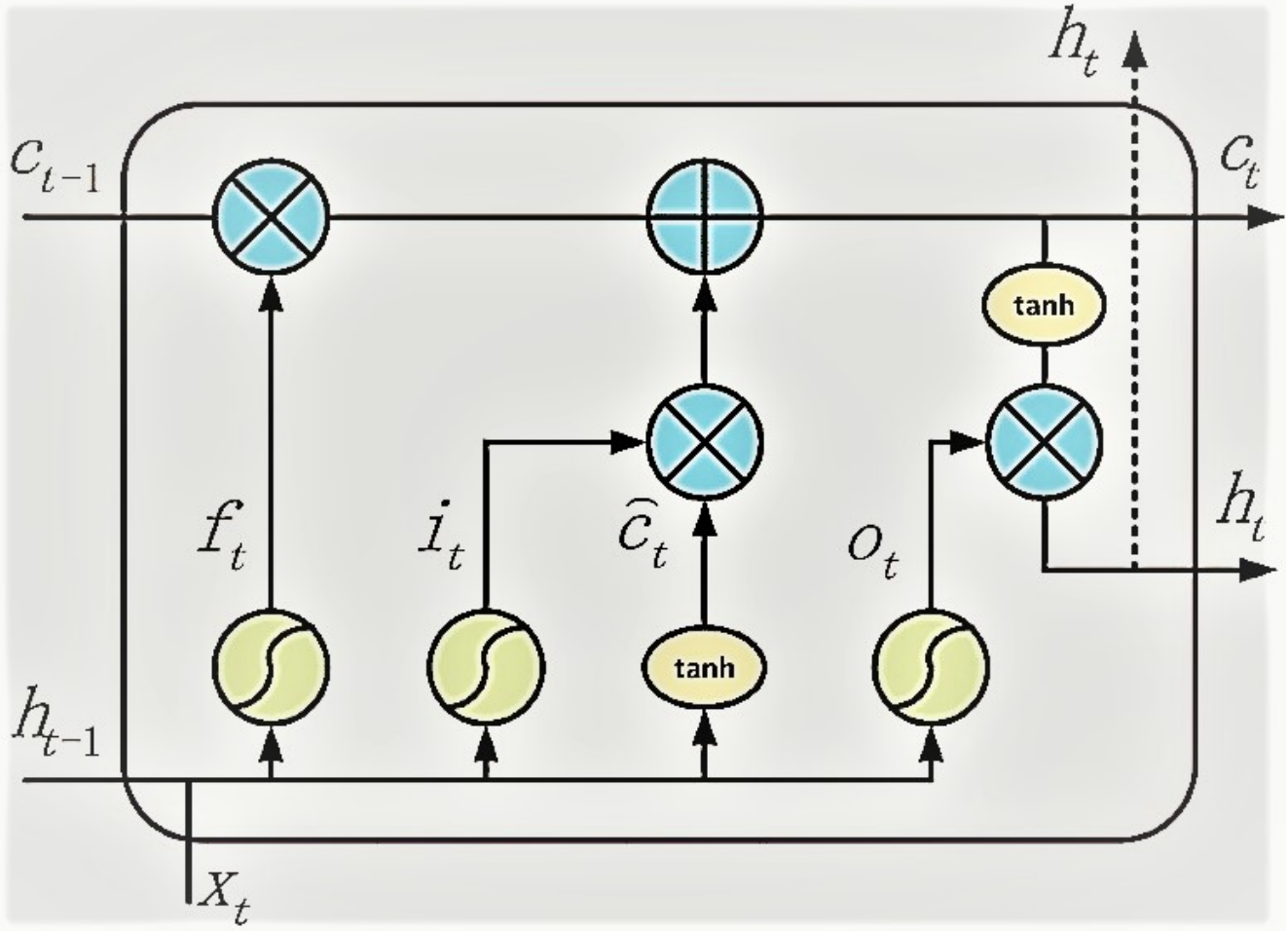

Develop RNN cell

```
import torch
import torch.nn as nn
from torch.autograd import Variable
import numpy as np

class LSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size, bias=True):
        super(LSTMCell, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.bias = bias

        self.xh = nn.Linear(input_size, hidden_size * 4, bias=bias)
        self.hh = nn.Linear(hidden_size, hidden_size * 4, bias=bias)
        self.reset_parameters()

    def reset_parameters(self):
        std = 1.0 / np.sqrt(self.hidden_size)
        for w in self.parameters():
            w.data.uniform_(-std, std)

    def forward(self, input, hx=None):
```

```
        # Inputs:
        #       input: of shape (batch_size, input_size)
        #       hx: of shape (batch_size, hidden_size)
        # Outputs:
        #       hy: of shape (batch_size, hidden_size)
        #       cy: of shape (batch_size, hidden_size)

        if hx is None:
            hx = Variable(input.new_zeros(input.size(0), self.hidden_size))
            hx = (hx, hx)

        hx, cx = hx

        gates = self.xh(input) + self.hh(hx)

        # Get gates (i_t, f_t, g_t, o_t)
        input_gate, forget_gate, cell_gate, output_gate = gates.chunk(4, 1)

        i_t = torch.sigmoid(input_gate)
        f_t = torch.sigmoid(forget_gate)
        g_t = torch.tanh(cell_gate)
        o_t = torch.sigmoid(output_gate)

        cy = cx * f_t + i_t * g_t

        hy = o_t * torch.tanh(cy)


        return (hy, cy)

class RNNCell(nn.Module):
    def __init__(self, input_size, hidden_size, bias=True, nonlinearity="tanh")
        super(RNNCell, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.bias = bias
        self.nonlinearity = nonlinearity
        if self.nonlinearity not in ["tanh", "relu"]:
            raise ValueError("Invalid nonlinearity selected for RNN.")

        self.x2h = nn.Linear(input_size, hidden_size, bias=bias)
        self.h2h = nn.Linear(hidden_size, hidden_size, bias=bias)

        self.reset_parameters()


    def reset_parameters(self):
        std = 1.0 / np.sqrt(self.hidden_size)
        for w in self.parameters():
            w.data.uniform_(-std, std)


    def forward(self, input, hx=None):
```

```
        # Inputs:
        #       input: of shape (batch_size, input_size)
        #       hx: of shape (batch_size, hidden_size)
        # Output:
        #       hy: of shape (batch_size, hidden_size)

        if hx is None:
            hx = Variable(input.new_zeros(input.size(0), self.hidden_size))

        hy = (self.x2h(input) + self.h2h(hx))

        if self.nonlinearity == "tanh":
            hy = torch.tanh(hy)
        else:
            hy = torch.relu(hy)

        return hy

class GRUCell(nn.Module):
    def __init__(self, input_size, hidden_size, bias=True):
        super(GRUCell, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.bias = bias

        self.x2h = nn.Linear(input_size, 3 * hidden_size, bias=bias)
        self.h2h = nn.Linear(hidden_size, 3 * hidden_size, bias=bias)

        self.reset_parameters()


    def reset_parameters(self):
        std = 1.0 / np.sqrt(self.hidden_size)
        for w in self.parameters():
            w.data.uniform_(-std, std)

    def forward(self, input, hx=None):

        # Inputs:
        #       input: of shape (batch_size, input_size)
        #       hx: of shape (batch_size, hidden_size)
        # Output:
        #       hy: of shape (batch_size, hidden_size)

        if hx is None:
            hx = Variable(input.new_zeros(input.size(0), self.hidden_size))

        x_t = self.x2h(input)
        h_t = self.h2h(hx)


        x_reset, x_upd, x_new = x_t.chunk(3, 1)
        h_reset, h_upd, h_new = h_t.chunk(3, 1)
```

```
        reset_gate = torch.sigmoid(x_reset + h_reset)
        update_gate = torch.sigmoid(x_upd + h_upd)
        new_gate = torch.tanh(x_new + (reset_gate * h_new))

        hy = update_gate * hx + (1 - update_gate) * new_gate

        return hy
```

Develop RNN model

```
import torch
import torch.nn as nn
from torch.autograd import Variable
import numpy as np

from rnncells import LSTMCell, GRUCell, RNNCell

class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, bias, output_size, a
        super(SimpleRNN, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.bias = bias
        self.output_size = output_size

        self.rnn_cell_list = nn.ModuleList()

        if activation == 'tanh':
            self.rnn_cell_list.append(RNNCell(self.input_size,
                                              self.hidden_size,
                                              self.bias,
                                              "tanh"))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(RNNCell(self.hidden_size,
                                                  self.hidden_size,
                                                  self.bias,
                                                  "tanh"))

        elif activation == 'relu':
            self.rnn_cell_list.append(RNNCell(self.input_size,
                                              self.hidden_size,
                                              self.bias,
                                              "relu"))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(RNNCell(self.hidden_size,
                                              self.hidden_size,
                                              self.bias,
                                              "relu"))
        else:
            raise ValueError("Invalid activation.")
```

```
        self.fc = nn.Linear(self.hidden_size, self.output_size)


    def forward(self, input, hx=None):

        # Input of shape (batch_size, seqence length, input_size)
        #
        # Output of shape (batch_size, output_size)

        if hx is None:
            if torch.cuda.is_available():
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h
            else:
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h

        else:
             h0 = hx

        outs = []

        hidden = list()
        for layer in range(self.num_layers):
            hidden.append(h0[layer, :, :])

        for t in range(input.size(1)):

            for layer in range(self.num_layers):

                if layer == 0:
                    hidden_l = self.rnn_cell_list[layer](input[:, t, :], hidden[
                else:
                    hidden_l = self.rnn_cell_list[layer](hidden[layer - 1],hidde
                hidden[layer] = hidden_l

                hidden[layer] = hidden_l

            outs.append(hidden_l)

        # Take only last time step. Modify for seq to seq
        out = outs[-1].squeeze()

        out = self.fc(out)


        return out

class LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, bias, output_size):
        super(LSTM, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.bias = bias
```

```
        self.output_size = output_size

        self.rnn_cell_list = nn.ModuleList()

        self.rnn_cell_list.append(LSTMCell(self.input_size,
                                           self.hidden_size,
                                           self.bias))
        for l in range(1, self.num_layers):
            self.rnn_cell_list.append(LSTMCell(self.hidden_size,
                                               self.hidden_size,
                                               self.bias))

        self.fc = nn.Linear(self.hidden_size, self.output_size)

    def forward(self, input, hx=None):

        # Input of shape (batch_size, seqence length , input_size)
        #
        # Output of shape (batch_size, output_size)

        if hx is None:
            if torch.cuda.is_available():
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h
            else:
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h
        else:
             h0 = hx

        outs = []

        hidden = list()
        for layer in range(self.num_layers):
            hidden.append((h0[layer, :, :], h0[layer, :, :]))

        for t in range(input.size(1)):

            for layer in range(self.num_layers):

                if layer == 0:
                    hidden_l = self.rnn_cell_list[layer](
                        input[:, t, :],
                        (hidden[layer][0],hidden[layer][1])
                        )
                else:
                    hidden_l = self.rnn_cell_list[layer](
                        hidden[layer - 1][0],
                        (hidden[layer][0], hidden[layer][1])
                        )

                hidden[layer] = hidden_l

            outs.append(hidden_l[0])

        out = outs[-1].squeeze()
```

```
        out = self.fc(out)

        return out

class GRU(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, bias, output_size):
        super(GRU, self).__init__()

        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.bias = bias
        self.output_size = output_size

        self.rnn_cell_list = nn.ModuleList()

        self.rnn_cell_list.append(GRUCell(self.input_size,
                                          self.hidden_size,
                                          self.bias))
        for l in range(1, self.num_layers):
            self.rnn_cell_list.append(GRUCell(self.hidden_size,
                                              self.hidden_size,
                                              self.bias))
        self.fc = nn.Linear(self.hidden_size, self.output_size)


    def forward(self, input, hx=None):

        # Input of shape (batch_size, seqence length, input_size)
        #
        # Output of shape (batch_size, output_size)

        if hx is None:
            if torch.cuda.is_available():
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h
            else:
                h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.h

        else:
             h0 = hx

        outs = []

        hidden = list()
        for layer in range(self.num_layers):
            hidden.append(h0[layer, :, :])

        for t in range(input.size(1)):

            for layer in range(self.num_layers):

                if layer == 0:
                    hidden_l = self.rnn_cell_list[layer](input[:, t, :], hidden[
```

```
                else:
                    hidden_l = self.rnn_cell_list[layer](hidden[layer - 1],hidde
                hidden[layer] = hidden_l

                hidden[layer] = hidden_l

            outs.append(hidden_l)

        # Take only last time step. Modify for seq to seq
        out = outs[-1].squeeze()

        out = self.fc(out)

        return out

class BidirRecurrentModel(nn.Module):
    def __init__(self, mode, input_size, hidden_size, num_layers, bias, output_s
        super(BidirRecurrentModel, self).__init__()
        self.mode = mode
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.bias = bias
        self.output_size = output_size

        self.rnn_cell_list = nn.ModuleList()

        if mode == 'LSTM':

            self.rnn_cell_list.append(LSTMCell(self.input_size,
                                               self.hidden_size,
                                               self.bias))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(LSTMCell(self.hidden_size,
                                                   self.hidden_size,
                                                   self.bias))

        elif mode == 'GRU':
            self.rnn_cell_list.append(GRUCell(self.input_size,
                                              self.hidden_size,
                                              self.bias))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(GRUCell(self.hidden_size,
                                                  self.hidden_size,
                                                  self.bias))

        elif mode == 'RNN_TANH':
            self.rnn_cell_list.append(RNNCell(self.input_size,
                                              self.hidden_size,
                                              self.bias,
                                              "tanh"))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(RNNCell(self.hidden_size,
                                                  self.hidden_size,
```

```
                                                    self.bias,
                                                    "tanh"))

        elif mode == 'RNN_RELU':
            self.rnn_cell_list.append(RNNCell(self.input_size,
                                              self.hidden_size,
                                              self.bias,
                                              "relu"))
            for l in range(1, self.num_layers):
                self.rnn_cell_list.append(RNNCell(self.hidden_size,
                                              self.hidden_size,
                                              self.bias,
                                              "relu"))
        else:
            raise ValueError("Invalid RNN mode selected.")

        self.fc = nn.Linear(self.hidden_size * 2, self.output_size)

    def forward(self, input, hx=None):

        # Input of shape (batch_size, sequence length, input_size)
        #
        # Output of shape (batch_size, output_size)

        if torch.cuda.is_available():
            h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.hidde
        else:
            h0 = Variable(torch.zeros(self.num_layers, input.size(0), self.hidde

        if torch.cuda.is_available():
            hT = Variable(torch.zeros(self.num_layers, input.size(0), self.hidde
        else:
            hT = Variable(torch.zeros(self.num_layers, input.size(0), self.hidde

        outs = []
        outs_rev = []

        hidden_forward = list()
        for layer in range(self.num_layers):
            if self.mode == 'LSTM':
                hidden_forward.append((h0[layer, :, :], h0[layer, :, :]))
            else:
                hidden_forward.append(h0[layer, :, :])

        hidden_backward = list()
        for layer in range(self.num_layers):
            if self.mode == 'LSTM':
                hidden_backward.append((hT[layer, :, :], hT[layer, :, :]))
            else:
                hidden_backward.append(hT[layer, :, :])

        for t in range(input.shape[1]):
            for layer in range(self.num_layers):
```

```
                if self.mode == 'LSTM':
                    # If LSTM
                    if layer == 0:
                        # Forward net
                        h_forward_l = self.rnn_cell_list[layer](
                            input[:, t, :],
                            (hidden_forward[layer][0], hidden_forward[layer][1])
                            )
                        # Backward net
                        h_back_l = self.rnn_cell_list[layer](
                            input[:, -(t + 1), :],
                            (hidden_backward[layer][0], hidden_backward[layer][1
                            )
                    else:
                        # Forward net
                        h_forward_l = self.rnn_cell_list[layer](
                            hidden_forward[layer - 1][0],
                            (hidden_forward[layer][0], hidden_forward[layer][1])
                            )
                        # Backward net
                        h_back_l = self.rnn_cell_list[layer](
                            hidden_backward[layer - 1][0],
                            (hidden_backward[layer][0], hidden_backward[layer][1
                            )

                else:
                    # If RNN{_TANH/_RELU} / GRU
                    if layer == 0:
                        # Forward net
                        h_forward_l = self.rnn_cell_list[layer](input[:, t, :],
                        # Backward net
                        h_back_l = self.rnn_cell_list[layer](input[:, -(t + 1),
                    else:
                        # Forward net
                        h_forward_l = self.rnn_cell_list[layer](hidden_forward[l
                        # Backward net
                        h_back_l = self.rnn_cell_list[layer](hidden_backward[lay


                hidden_forward[layer] = h_forward_l
                hidden_backward[layer] = h_back_l

            if self.mode == 'LSTM':

                outs.append(h_forward_l[0])
                outs_rev.append(h_back_l[0])

            else:
                outs.append(h_forward_l)
                outs_rev.append(h_back_l)

        # Take only last time step. Modify for seq to seq
        out = outs[-1].squeeze()
        out_rev = outs_rev[0].squeeze()
```

```
        out = torch.cat((out, out_rev), 1)

        out = self.fc(out)
        return out
```